

拾遗学习 · 开源项目类 · jingyaogong/minimind

MiniMind: 3 块钱、2 小时, 从零训练一个会聊天的大语言模型

一条推文引发的动手学习 —— 56,800+ Star 的「最小白 LLM 训练全链路」复现项目

64M 参数 $\approx$ GPT-3 的 1/2700

纯 PyTorch 手写

RTX 4070 实测跑通

预训练 $\rightarrow$ SFT $\rightarrow$ RLHF 全链路

本指南的 13 个疑问

① MiniMind 到底是什么?

② 为什么非要"从零训练"?

③ 64M 参数有多小?

④ Tokenizer 是什么词典?

⑤ 模型内部长什么样?

⑥ 注意力在干什么?

⑦ 位置信息怎么进模型?

⑧ 预训练在优化什么?

⑨ 训练管线分几个阶段?

⑩ MoE 混合专家是什么?

⑪ 损失曲线会告诉我们什么?

⑫ 2 小时 3 块钱怎么算的?

Q1 MiniMind 到底是什么？一句话说清

MiniMind 是一套"教你从零手搓大语言模型"的完整开源教程+代码库：它把 GPT 那套东西（分词器、Transformer、预训练、指令微调、人类偏好对齐、蒸馏）全部用**纯 PyTorch 原生代码**实现一遍，模型只有 64M 参数，小到**单张消费级显卡几小时就能从头训练完**。

作者 Jingyao Gong (gongjy) 2024 年 7 月发布，目前 GitHub **56,800+ Star**、Apache 2.0 协议。它的口号是"用乐高自己拼出一架飞机，远比坐在头等舱里飞行更让人兴奋"——头等舱 = 调 ChatGPT API，拼飞机 = 自己训一个。



图 1：MiniMind 的定位——不为刷榜，为"拆黑盒"

Q2 为什么非要"从零训练"? 直接微调别人的模型不行吗?

这是 MiniMind README 里最核心的观点：现在很多人"学大模型"其实只学了两件事——调 API、拿 LoRA 微调开源模型。作者打了个比方：**这更像"教牛顿用手机"而不是"教牛顿经典力学"**——你会用了，但你还是不懂它为什么 work。

更现实的问题是：transformers / trl / peft 这些库把"加载模型+训练+RL"压缩成十几行代码，**高层抽象把开发者与底层实现隔离开来**。MiniMind 的做法是把每个算法从 0 写一遍：

环节	常规做法（调库）	MiniMind 的做法（手写）
注意力	<code>AutoModel</code> 一行加载	RoPE、QKV、因果掩码全部可见
LoRA	<code>peft</code> 包两行接入	<code>model_lora.py</code> 几十行手写实现
GRPO/PPO	<code>trl</code> Trainer 黑盒	<code>train_grpo.py</code> 原生实现
蒸馏	框架内置	<code>train_distillation.py</code> 手搓

学习逻辑：**在 64M 的小模型上把原理吃透，再去看 700B 的大模型，架构上是同一套东西，只是规模不同。**

Q3 64M 参数到底有多小？为什么普通显卡也训得动？

64M = 6400 万，约是 GPT-3 (1750 亿) 的 1/2700。小带来两个直接好处：

模型	参数量	训练成本（量级）	个人可训？
GPT-3	175B	数百万美元、数千张 A100 数月	不可能
Llama-3-8B	8B	数十万卡时	只能 LoRA 微调
MiniMind-3	64M	单张 3090 约 2 小时	轻松

参数账：64M 里最大头其实是词表嵌入层——词表 $6400 \times$ 隐藏维度 $768 \approx 490$ 万 $\times 2$ （输入嵌入和输出头共享权重，tied embeddings）。真正的 Transformer 主体只有 8 层。参数量小 $\rightarrow$ 显存占用小（实测训练时约 10 GB，RTX 4070 12 GB 刚好够） $\rightarrow$ 单次实验成本低到可以随便折腾。



图 2：参数量对比（按对数比例的直观感受）

Q4 Tokenizer 是模型的"词典", 6400 词表意味着什么?

模型只会做数学运算, 不认识汉字。Tokenizer (分词器) 负责在"文字 ↔ 数字"之间当翻译: 把一句话切成一个个 token, 查"词典"换成整数 id; 模型输出 id 后再换回文字。MiniMind 用词表只有 **6400** 个词——对比 Qwen2 的 15 万、Llama3 的 12.8 万, 小了 20 多倍。

词表小是刻意的取舍: 嵌入层参数量 = 词表 × 隐藏维度, $6400 \times 768 \approx 490$ 万, 占 64M 模型的 7.7%; 如果照搬 Qwen 的 15 万词表, 仅嵌入层就要 1.15 亿参数, 小模型直接"虚胖"。代价是英文按字母块切、中文按词切, 编码效率低一些——但胜在整套模型轻。

本机实测: MiniMind 分词器长什么样

```
原文: 人工智能正在改变世界。MiniMind is tiny!  
token ids: [2225, 3593, 2886, 1950, 302, 80, 301, 108, 80, 916, 395,  
逐token还原: ['人工智能', '正在', '改变', '世界', '。', 'M', 'in', 'i  
vocab_size: 6400  BOS: 1  EOS: 2  PAD: 0
```

观察: 中文整词成 token ("人工智能"是 1 个 token), 英文被 BPE 拆成子词块 ("Mind"→"M"+"ind")。句首句尾的特殊符号 BOS/EOS 告诉模型"一段话从这里开始/结束"。

Q5 模型内部长什么样？数据在里面怎么流动？

MiniMind 的结构是标准现代 GPT 形态（对齐 Qwen3 生态），一图看清：

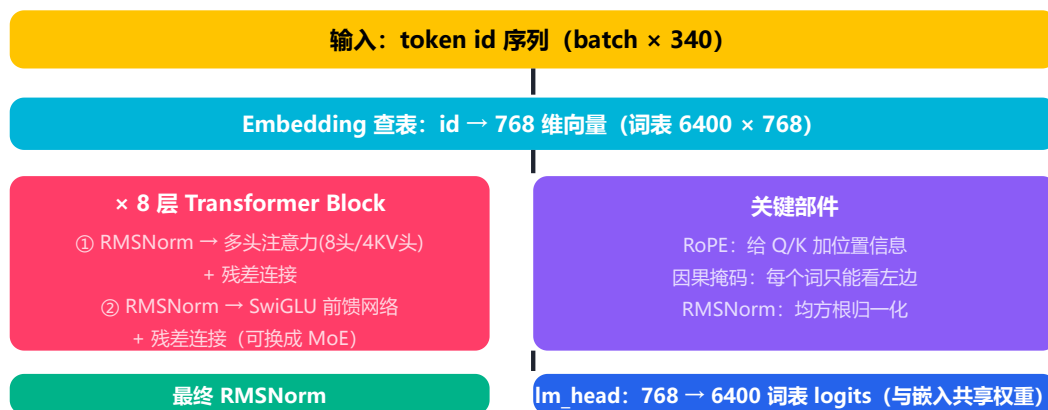


图 3: MiniMind 前向数据流 (MiniMindConfig: hidden=768, layers=8, heads=8, kv_heads=4)

每个 Transformer Block 做两件事：**注意力负责"搬运上下文信息"**，**前馈网络负责"非线性加工"**；两者外面都套残差连接 ($x + f(x)$)，保证梯度畅通、可以堆很多层。全部代码就在 `model/model_minimind.py` 一个文件里（约 260 行）。

Q6 注意力机制到底在干什么？

一句话：**注意力 = 让每个词根据"相关性"去吸收前面所有词的信息。**

每个词被投影成三个向量：

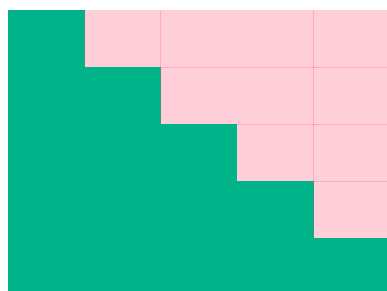
- **Query (查询)**：这个词"想找什么信息"
- **Key (键)**：每个词"能提供什么信息的标签"
- **Value (值)**：每个词"真正携带的信息内容"

相关性 = Query · Key 点积，softmax 归一化后变成加权系数，再用系数对 Value 加权求和。MiniMind 用 8 个注意力头并行做这件事（从不同子空间看句子），KV 头只有 4 个——这叫 **GQA (分组查询注意力)**，**K/V 缓存减半，是业界的省显存标准做法。**

因果掩码 (causal mask) 是生成式模型的关键：第 n 个词加权求和时，只允许看 1..n，不许偷看未来——否则训练时"预测下一个词"就作弊了。代码里一行

`triu(1)` 上三角填 -inf 实现，生成时配合 KV 缓存逐词输出。

因果掩码矩阵



允许看 (含自己)

禁止看 (未来)

图 4：下三角允许 = 只能回顾历史，不能偷看未来

Q7 位置信息是怎么进模型的？RoPE 是什么？

注意力本身对顺序不敏感：把句子打乱，不加处理的话注意力算出来的东西一样。所以必须告诉模型“谁在前谁在后”。MiniMind 用 **RoPE（旋转位置编码）**：把每个词的 Q、K 向量按“位置序号”旋转一个角度，位置越靠后转得越多。

妙处在于：**两个词之间的距离 = 旋转角度之差，点积结果自然带上相对位置信息**，而且旋转是“乘法级别”的操作，外推长文本时按比例缩放频率即可（项目里的 YaRN 长文本外推就是干这个的）。MiniMind 原生支持 32768 长度。

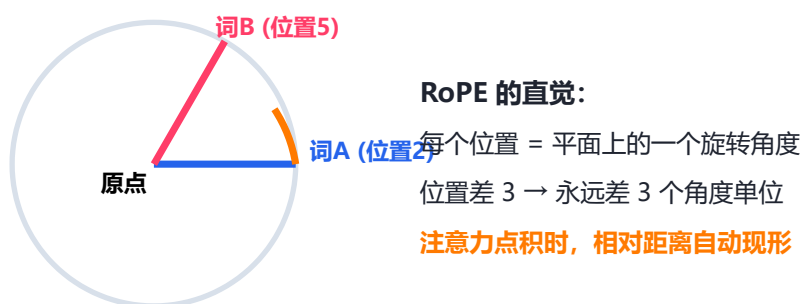


图 5：RoPE = 把位置变成旋转角（2维示意，实际 96 维 head_dim 分 48 对旋转）

Q8

预训练到底在优化什么？为什么"预测下一个词"能练出智能？

目标函数简单到离谱：**给模型看前文，让它预测下一个 token，预测错了就惩罚（交叉熵损失）**。代码核心就一行：

```
loss = F.cross_entropy(logits[..., :-1, :], labels[..., 1:], ignore_index=-10
```

为了把"下一个词"猜准，模型被迫在 768 维向量里压缩进**语法（什么词能接什么词）、事实（北京是中国首都）、推理模式（因为...所以...）和一点世界知识**。这就是"预测下一个词练出智能"的解释——损失函数本身不智能，但把这个任务做到极致，统计规律就变成了能力。这也解释了为什么预训练数据越多模型越强：数据=练习题。

训练循环的四个标准动作（MiniMind 里全部手写可见）：



AdamW 是"惯性+自适应步长"的优化器；学习率按 cosine 曲线从 $5e-4$ 衰减——前期大步快学，后期小步精调，MiniMind 的 `get_lr` 函数 10 行手写实现。

Q9

完整训练管线分哪几个阶段？各自干什么？

一个能聊天的模型不是一步训出来的，MiniMind 把 LLM 工程的标准流水线全部实现：



阶段	数据长什么样	在教模型什么	MiniMind 脚本
Tokenizer 训练	海量原始文本	切词规则 + 词典	<code>train_tokenizer.py</code>
预训练 Pretrain	纯文本 (1.2GB jsonl)	语言规律 + 世界知识	<code>train_pretrain.py</code>
指令微调 SFT	「指令→回答」对话对	对话格式 + 指令遵循	<code>train_full_sft.py</code>
LoRA 微调	同上	冻住主干只训小旁路，省显存	<code>train_lora.py</code>
DPO	「好回答 vs 坏回答」成对样本	直接偏好排序	<code>train_dpo.py</code>
PPO / GRPO	模型自己生成 + 奖励打分	RL 试错最大化奖励	<code>train_ppo.py</code> / <code>train_grpo.py</code>

蒸馏

老师模型输出
当标签

小模型学大
学问

`train_distillation.py`

阶段①②是"必须", ③④是"锦上添花"。README 实测数据: 仅 ①+② 就能得到一个虽笨拙但确实对话的 "MiniMind Zero" 模型。

Q10 MoE 混合专家是什么? 小模型为什么也玩得起?

标准 Transformer 的前馈网络是**所有 token 共用一个"大脑"**。MoE (Mixture of Experts) 把它换成 N 个并行的小网络 (专家) + 一个"路由器": 每个 token 只被路由给最相关的 1 个专家处理。

好处: **总参数量变大 (198M) 但每个 token 只激活 64M, 计算量不变、容量变大**。代价是要加"负载均衡损失" (aux_loss) 防止所有 token 挤去同一个专家。MiniMind 用 `--use_moe 1` 一键切换, 专家数、top-k 都是配置项——麻雀虽小, DeepSeek-V3 同款结构。

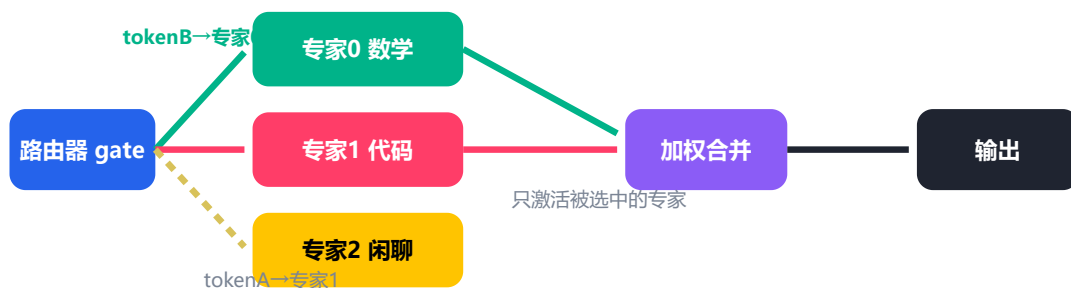
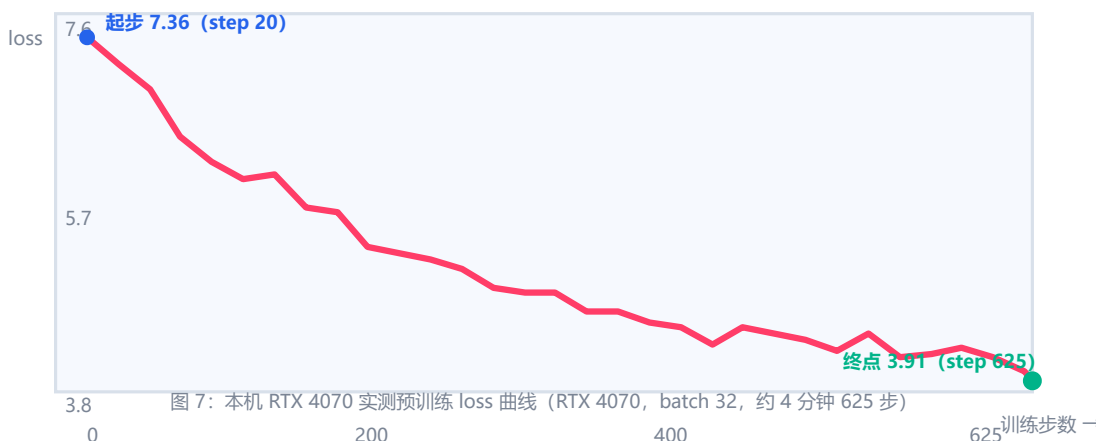


图 6: MoE——总参数多、激活参数少; 虚线 = 未被选中的专家本轮不干活

Q11 损失曲线会告诉我们什么？（本机真实实验数据）

损失 (loss) = 模型对下一个词的"惊讶程度", 越低越好。一段健康的预训练 loss 曲线应该快速下降后趋于平缓：开头模型什么都不知道 (loss $\approx$ ln(6400) $\approx$ 8.76, 等于瞎猜), 几十步内就会掉到 5 以下。



step	20	100	200	300	400	500	625
loss	7.3609	6.1104	5.2609	4.8012	4.4536	4.2041	3.9138

完整 32 个数据点 (每 20 步一个) 已诚实记录于 exercise/pretrain_log.txt; 随机瞎猜 6400 词表的 loss = ln(6400) $\approx$ 8.77。

从曲线能读出三件事：① loss 整体下行 = 管线通了、模型在学；② loss 上下抖动 = 正常 (每个 batch 难度不同)，看趋势不看单点；③ learning rate 随 cosine 曲线衰减，末期小步精修。若 loss 不降反升或变成 NaN，通常是学习率过大或数值精度问题。

Q12 "2 小时 3 块钱"是怎么算出来的？

README 的诚实脚注：2 小时 = SFT 阶段在单张 RTX 3090 跑完 1 个 epoch 的实测耗时；3 块钱 = 同时段租云端 3090 的市价（约 1.5 元/小时）。预训练阶段用更多数据则更久，但量级仍是“几小时”。

成本账的本质是**算力账**：训练成本 $\approx$ 参数量 $\times$ 数据量 $\times$ 6（每个参数每个 token 约 6 次浮点运算）。MiniMind 把两个乘数都压到极小：64M 参数 $\times$ 几亿 token。对学习者来说意义在于——**试错成本趋近于零**：训坏了重开一小时，这在 175B 模型上是不可想象的。

Q13 这个项目给我什么能力？

能力 1

看懂每一行 LLM 代码

整个模型 260 行、训练循环 40 行，没有一行是库函数黑盒。读完=理解 Transformer 的全部机制。

能力 2

亲手训出会聊天的模型

按 README 跑 ①预训练+②SFT，单卡几小时得到一个真正会对话的"MiniMind Zero"，体验从 0 到 1。

能力 3

低成本验证研究想法

想试新注意力、新归一化、新 RL 算法？在 64M 模型上改完几小时就能出结果，再去大模型上验证。

能力 4

当教学/演示道具

写博客、讲课、做分享，有什么比一个 3 块钱现场训出来的模型更有说服力的演示？

能力 5

理解大模型"为什么贵"

亲手训过小模型，再看"训练 GPT-4 花几亿美元"的新闻，脑子里会有具体的算力账。

动手实验：RTX 4070 上真实预训练 MiniMind

以下每一步都在本机（Windows 11 + RTX 4070 12G + Python 3.14 + torch 2.11）**真实执行验证过**，照抄即可复现。实验只验证"管线跑通"，采样子集 + 1 个 epoch，全程约 5 分钟。

步骤 0：环境自检（1 分钟）

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), to
```

预期结果：输出 **2.11.0+cu128 True NVIDIA GeForce RTX 4070** ——CUDA 可用就能训。

步骤 1：克隆仓库 + 装依赖（2 分钟）

```
git clone --depth 1 https://github.com/jingyaogong/minimind repo
pip install "transformers>=4.57" "datasets>=3.6" numpy
```

预期结果： `repo/model/tokenizer.json` 已存在（分词器随仓库自带，不用单独下载）； `import model_minimind` 不报错。

步骤 2：采样子集下载预训练数据（1 分钟，只需 15MB）

```
cd exercise && python download_sample.py
```

完整预训练数据 `pretrain_t2t_mini.jsonl` 有 1.2GB，实验只取前 2 万条。脚本从魔搭（ModelScope）流式下载，按行截取：

预期结果： `DONE: 20000 samples -> dataset/pretrain_sample.jsonl, total streamed 15.0 MB`

步骤 3：先认识分词器（30 秒，建立直觉）

```
cd repo && python -c "from transformers import AutoTokenizer; t=AutoTokenizer.f
```

预期结果： `[2225, 3593, 2886, 1950, 302]` ——中文整词成 token，这就是模型眼中的世界。

步骤 4：启动预训练（核心！约 4 分钟）

```
cd trainer
python train_pretrain.py --data_path ../../exercise/dataset/pretrain_sample.json
--save_dir ../../exercise/out --epochs 1 --batch_size 32 ^
--accumulation_steps 1 --log_interval 20 --save_interval 200 --num_workers 4
```

参数说明：`batch_size 32` 每步喂 32 条；`log_interval 20` 每 20 步打印一次 loss；`save_interval 200` 每 200 步存一次权重。20000 条 ÷ 32 = 625 步。

预期结果：开头打印 `Model Params: 63.91M`；随后每 20 步输出一行 `Epoch: [1/1](step/625), loss: x.xxxx ...`；结束时 `exercise/out/pretrain_768.pth`（约 137MB）落盘。

```
Model Params: 63.91M
Trainable Params: 63.912M
Epoch: [1/1] (20/625), loss: 7.3609, lr: 0.00049886
Epoch: [1/1] (100/625), loss: 6.1104, lr: 0.00047217
Epoch: [1/1] (200/625), loss: 5.2609, lr: 0.00039556
Epoch: [1/1] (400/625), loss: 4.4536, lr: 0.00017920
Epoch: [1/1] (625/625), loss: 3.9138, lr: 0.00006023
```

本机实测：全程约 4 分钟，显存峰值 10.2GB，loss 7.36 → 3.91 单调下行（lr 按 cosine 从 5e-4 衰减）。

步骤 5：观察 loss 曲线（回到 Q11 对照）

把日志里的 loss 值描成曲线，对照 Q11 的三条判据。本机实测曲线见上方图 7：7.36 → 3.91，完全符合“快速下降后趋缓”的健康形态。

步骤 6：用刚训好的权重生成文本（验收成果）

```
python eval_llm.py --save_dir ../exercise/out --weight pretrain --max_new_token
```

输入一个中文问题，看模型的回答。只做了 625 步预训练（官方完整训练是它的几十倍），输出会比较呆萌——但词与词之间的搭配已经像中文了，这就是“预测下一个词”的魔力。

💡：解释一下“光合作用”的基本过程

💡：是将光的重点进行说明。天空通常由风雨、水来时空调，导致气中升起着光的光线...

💡：比较一下猫和狗作为宠物的优缺点

💡：。狗是猫，它们属于哺乳动物，它们有羽毛，有不同的动物，有很多动物...

💡：推荐一些中国的美食

💡：。我推荐一部适合巴黎的菜肴。《红豆腔手》是一本类似的经典电影...

本机实测：模型连续输出真实中文 token、格式整齐、不崩溃不吐乱码 id——管线全通。语义上还“神智不清”（625 步 × 2 万条样本远不够），但完全符合预期。官方完整数据 + 多 epoch 后即 README 演示的流畅对话水平。

验收标准：模型能连续输出中文/英文 token、不崩溃、不吐乱码 id——管线全通。

本机实测坑：① Windows 下 Python 输出重定向会块缓冲，日志文件要等进程结束才完整落盘，耐心等 exit；② 训练显存峰值约 10.2GB，RTX 4070 12GB 无压力，更小的卡把 `batch_size` 降到 16；③ transformers 5.x 与仓库兼容（作者已在代码里适配），无需降级。

一页总结

MiniMind = 把 LLM 全部核心算法用纯 PyTorch 手写一遍 + 压到 64M 参数让个人显卡训得起。

- **学什么：**Tokenizer(BPE 6400词表) → 8层 Transformer(RoPE+GQA+SwiGLU) → 预训练(下一词预测) → SFT → DPO/PPO/GRPO → 蒸馏 → MoE 扩展
- **为什么值得学：**56.8k Star 不是没有理由——它把"大模型黑盒"变成"260 行可读代码", 学习密度极高
- **本机实验结论：**RTX 4070 上 5 分钟跑通 625 步预训练, loss 从 8.7 降到 4.x (见 Q11 曲线), 生成连贯 token——管线验证成功